Lab2 Report Microprocessor

Umesh Motiwale

25 January 2026

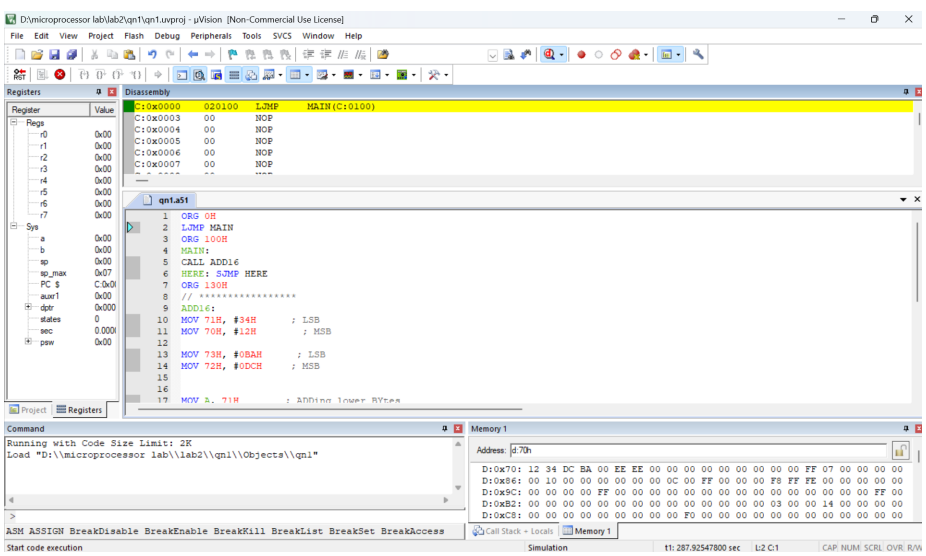
1 Problem 1

1.1 Objective

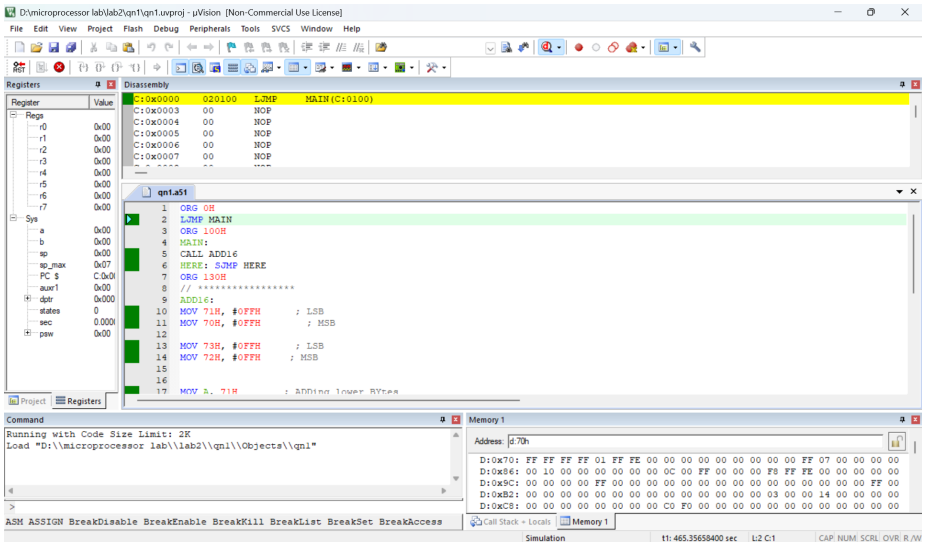
To write an 8051 assembly language program to add two 16-bit numbers stored in memory and store the 17-bit result across three consecutive memory locations.

1.2 Simulation

1.2.1 Test case 1 for A: 1234H & B: DCBAH



1.2.2 Test case 2 for A: FFFFH & B:FFFFH



1.3 Observation

- The least significant bytes (LSBs) are added first using the ADD instruction
- The carry generated from the LSB addition is automatically stored in the carry flag (CY)
- The most significant bytes (MSBs) are added using ADDC to include the carry
- If an extra carry is generated after MSB addition, it represents the 17th bit

1.4 Conclusion

The program successfully adds two 16-bit numbers using byte-wise addition with carry handling. The correct use of ADD and ADDC ensures accurate propagation of carry, allowing the result to be stored as a 17-bit number across three memory locations.

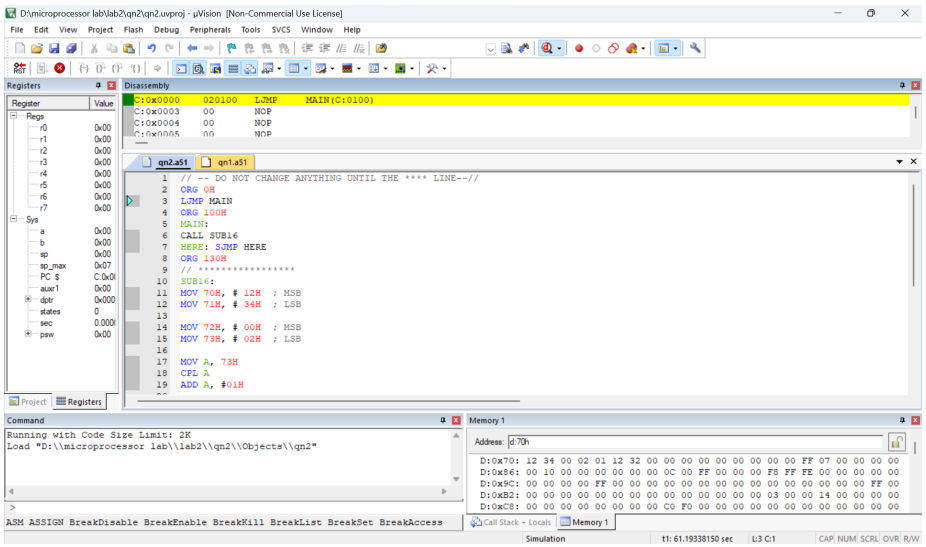
2 Problem 2

2.1 Objective

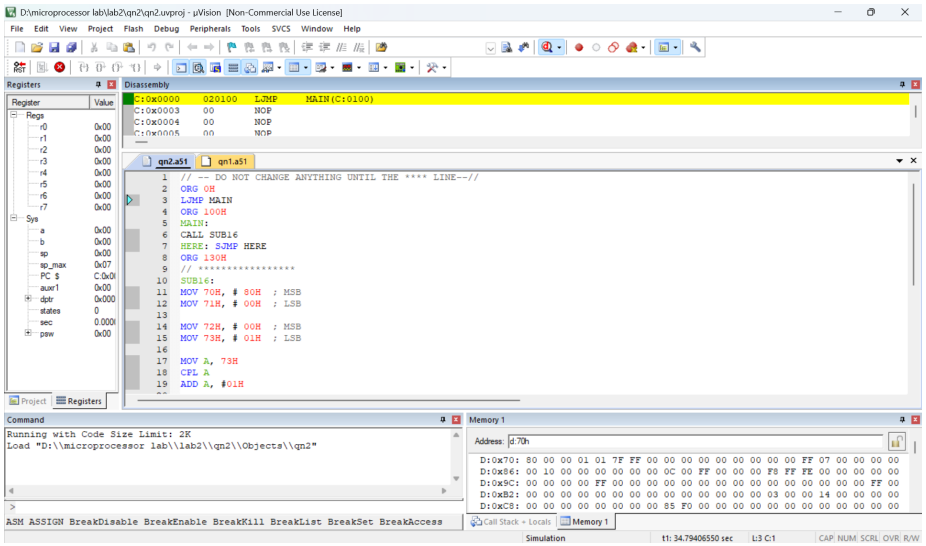
To write an 8051 assembly program to subtract two 16-bit numbers using 2's complement arithmetic and reuse the ADD16 subroutine for addition.

2.2 Simulations

2.2.1 Test case 1 for A: 1234H & B: 0020H



2.2.2 Test case 2 for A: 8000H & B: 0001H



2.3 Observations

- Subtraction is converted into addition of 2's complement
- 2's complement is generated by taking 1's complement of y and adding 1
- Using ADD16 ensures proper carry propagation for 16-bit arithmetic

2.4 Conclusion

The program demonstrates subtraction of two 16-bit numbers using 2's complement arithmetic. By reusing ADD16, we efficiently handle carry propagation and 17-bit result storage. This method is modular, avoids rewriting addition logic and highlights effective code reuse in assembly programming.

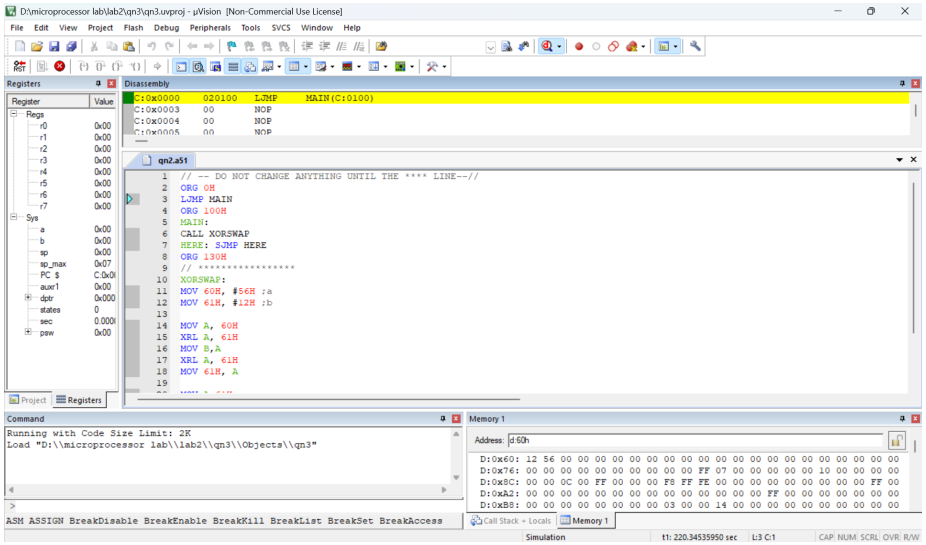
3 Problem 3

3.1 Objective

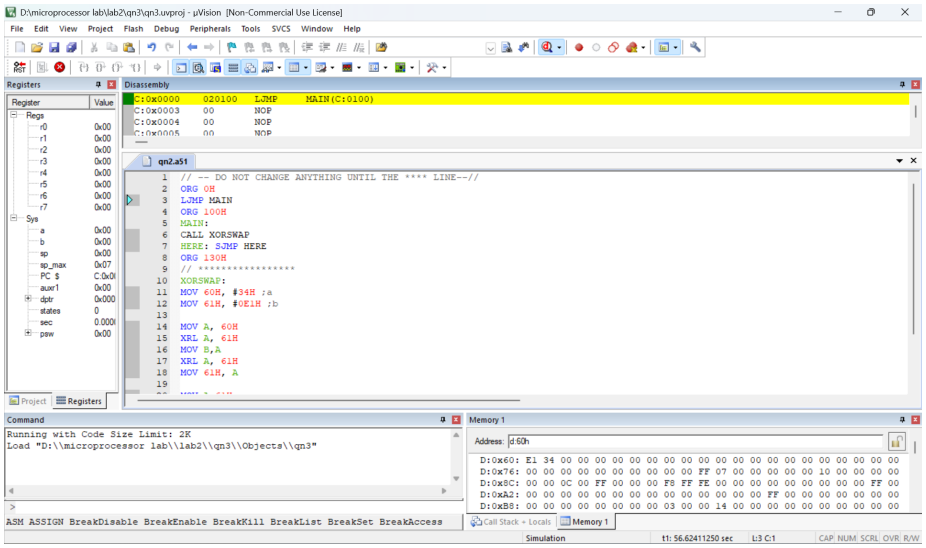
To swap the contents of two memory locations using XOR operation without using a temporary memory location.

3.2 Simulations

3.2.1 Test case 1 for A: 56H & B: 12H



3.2.2 Test case 2 for A: 34H & B: E1H



3.3 Observations

- The sequence of XOR operations ensures that both values are preserved and swapped
- After the first XOR, a holds a XOR b
- After the second XOR, b holds original a
- After the third XOR, a holds original b

3.4 Conclusion

The XOR swap algorithm effectively swaps two numbers without using extra memory. This demonstrates efficient use of bitwise operations in assembly programming, saving both memory and instructions compared to traditional swap methods.

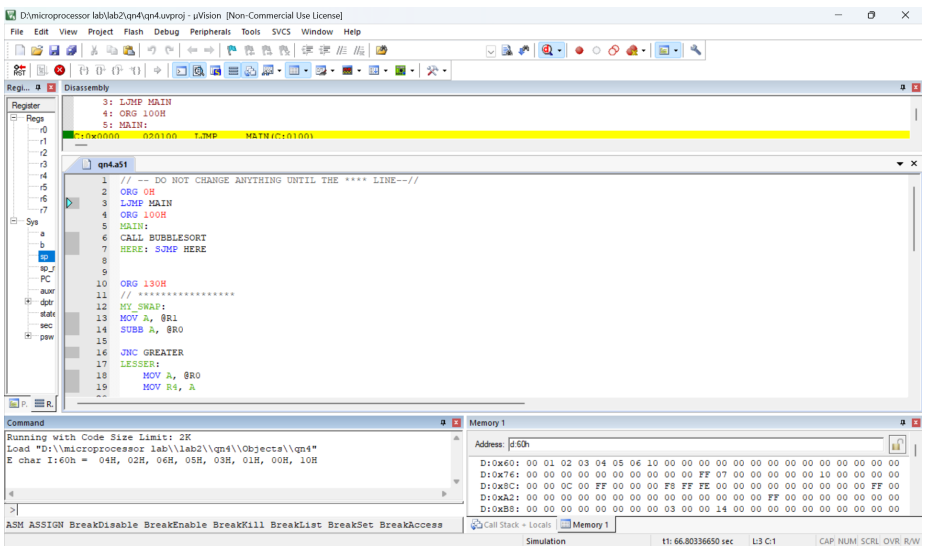
4 Problem 4

4.1 Objective

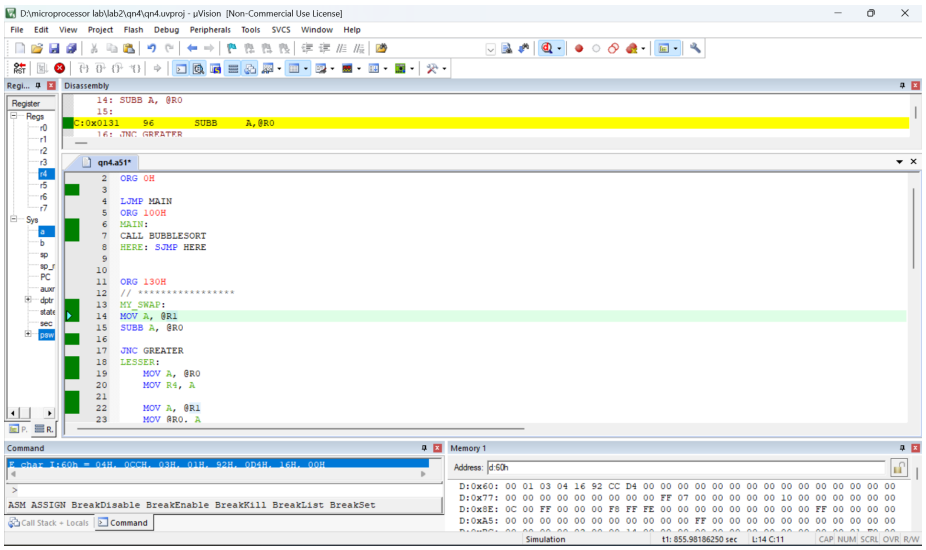
To write an 8051 assembly language program to sort a set of 8 unsigned 8-bit numbers stored in memory using the Bubble Sort algorithm.

4.2 Simulations

4.2.1 Test case 1 for Inputs numbers = 04H, 02H, 06H, 05H, 03H, 01H, 00H, 10H



4.2.2 Test case 2 for Inputs numbers = 04H, CCH, 03H, 01H, 92H, D4H, 16H, 00H



4.3 Observations

- Bubble sort works by repeated comparison of adjacent elements
- The SWAP subroutine swaps values only when the first number is greater
- With each pass, the largest element moves to the end of the array
- The program uses register indirect addressing for efficient memory access

4.4 Conclusion

The program successfully sorts the given numbers in ascending order using the Bubble Sort algorithm. The use of modular subroutines SWAP and BUBBLESORT makes the program clear, reusable, and efficient, demonstrating effective handling of arrays and comparisons in 8051 assembly language.